Detección de fraude en transacciones con tarjeta de crédito mediante aprendizaje de máquina

1st Lisbeth Cataño Sepulveda
Departamento de Ingenieria en Sistemas
Universidad de Antioquia UdeA
Medellin, Colombia
lisbeth.catano@udea.edu.co

2nd Kelly Julieth Arango Henao
Departamento de Ingenieria en Sistemas
Universidad de Antioquia UdeA
Medellin, Colombia
kjulieth.arango@udea.edu.co

3rd Maria Bernarda Salazar Sanchez
Departamento de Ingenieria en Sistemas
Universidad de Antioquia UdeA
Medellin, Colombia
bernarda.salazar@udea.edu.co

***Abstract*—Este trabajo presenta el diseño, análisis y simulación de un sistema de detección de fraude en transacciones con tarjeta de crédito mediante técnicas de aprendizaje automático, utilizando la base de datos pública Credit Card Fraud Detection, proporcionada por el Machine Learning Group de la Université Libre de Bruxelles (ULB). Después de realizar una limpieza inicial, el conjunto de datos utilizado contiene 283.726 transacciones, de las cuales 473 corresponden a fraudes. Debido al marcado desbalance entre las clases, se consideran diferentes métricas de evaluación además de la exactitud. Se presenta la descripción del problema, el análisis exploratorio de los datos, el estado del arte, la aproximación de aprendizaje automático seleccionada y el entrenamiento y evaluación de cinco modelos de clasificación, usando el área bajo la curva Precision-Recall como métrica principal.**

***Index Terms*—detección de fraude, aprendizaje automático, clases desbalanceadas, clasificación binaria, reducción de dimensionalidad**

I. Introducción

El fraude en las transacciones realizadas con tarjetas de crédito constituye un problema que puede generar pérdidas económicas tanto para los usuarios como para las entidades financieras. Actualmente se realizan miles de transacciones diariamente, por lo que su revisión manual resulta poco práctica y difícil de realizar a gran escala.

Frente a esta situación, el aprendizaje automático se presenta como una alternativa útil, ya que permite analizar grandes volúmenes de transacciones e identificar patrones que pueden ayudar a distinguir entre operaciones legítimas y potencialmente fraudulentas, sin depender exclusivamente de reglas definidas manualmente. El problema se plantea como una tarea de clasificación binaria supervisada, en la que cada transacción pertenece a una de dos clases: legítima o fraudulenta.

Uno de los principales retos al abordar este problema es el marcado desbalance entre las clases. La gran mayoría de las transacciones son legítimas y solo una proporción muy pequeña corresponde a operaciones fraudulentas. En este contexto, un modelo puede presentar una alta Accuracy general y, al mismo tiempo, tener un bajo desempeño en la detección de los casos de fraude, que constituyen la clase de mayor interés para el problema. Por esta razón, además de Accuracy, se consideran métricas como Precision, Recall, F1-score y ROC-AUC.

En este proyecto se aborda la detección de fraude mediante aprendizaje automático utilizando el conjunto de datos público Credit Card Fraud Detection, publicado por el Machine Learning Group de la Université Libre de Bruxelles (ULB) [1]. Se realiza la caracterización del conjunto de datos, el análisis exploratorio, la revisión del estado del arte, la definición de la aproximación de aprendizaje, y el entrenamiento y evaluación de varios modelos de clasificación.

II. Material y Métodos

A. Conjunto de datos

Para el desarrollo del trabajo se utiliza el conjunto de datos público Credit Card Fraud Detection, publicado por el Machine Learning Group de la Université Libre de Bruxelles (ULB) [1]. El conjunto de datos original contiene 284.807 transacciones, de las cuales 492 corresponden a operaciones fraudulentas.

Como parte del preprocesamiento inicial, se eliminaron 1.081 registros duplicados, obteniendo un conjunto final de 283.726 transacciones. De estas, 283.253 corresponden a transacciones legítimas y 473 a transacciones fraudulentas, lo que representa aproximadamente el 99,83% y el 0,17% respectivamente, evidenciando un fuerte desbalance entre las clases.

TABLE I
Resumen del conjunto de datos

Característica	Valor
Número de instancias	283.726 (tras eliminar 1.081 duplicados)
Variables de entrada	30 (Time, V1-V28, Amount)
Variable objetivo	Class (Legítima / Fraudulenta)
Distribución de clases	283.253 Legítima (99,83%) / 473 Fraudulenta (0,17%)
Datos faltantes	0
Tipo de datos	Numérico (V1-V28 ya transformadas con PCA)

En cuanto a las variables utilizadas, la base de datos contiene 31 columnas: 30 variables de entrada (Time, V1 a V28 y Amount) y una variable objetivo (Class). La variable Class toma el valor 0 para transacciones legítimas y 1 para transacciones fraudulentas. La variable Time representa el tiempo transcurrido desde la primera transacción, y Amount

corresponde al valor de la transacción. No se identificaron valores faltantes en ninguna variable.

TABLE II
VARIABLES DE LA BASE DE DATOS

Var.	Descripción	Tipo	Preproc.
Time	Segundos desde la primera transacción	Numérica	Estandarización
V1-V28	Componentes PCA anonimizados	Numérica	Ninguno
Amount	Monto de la transacción	Numérica	Estandarización
Class	0 = Legítima, 1 = Fraudulenta	Binaria	Ninguno

Las variables V1 a V28 corresponden a componentes obtenidos mediante un proceso de Análisis de Componentes Principales (PCA) aplicado por los autores originales del conjunto de datos para anonimizar la información [1]. Esta particularidad tiene implicaciones importantes para las siguientes etapas del proyecto. En primer lugar, dado que estas variables ya fueron transformadas y escaladas implícitamente por el PCA original, una segunda estandarización sobre ellas puede ser innecesaria, o incluso contraproducente para algunos modelos. En segundo lugar, la interpretabilidad de la importancia de cada variable es limitada, ya que no representan directamente una característica original de la transacción. En tercer lugar, no es posible aplicar PCA nuevamente de forma directa sobre estas variables, ya que equivaldría a realizar un PCA sobre un PCA. Por esta razón, en la etapa de reducción de dimensión del proyecto se define una estrategia específica que tenga en cuenta esta particularidad, considerando únicamente Time y Amount como candidatas a transformación adicional.

Cabe aclarar que, si bien las variables V1 a V28 no están en una escala comparable entre sí (su desviación estándar varía entre 1,95 en V1 y 0,33 en V28), esta diferencia de varianza no es un defecto, sino que refleja directamente la importancia relativa de cada componente en la transformación PCA original: los primeros componentes capturan más varianza de los datos, y por eso tienen una escala mayor. Reescalarlas artificialmente eliminaría esa información y les daría a todas el mismo peso relativo, independientemente de cuánta varianza explican. Por esta razón se optó por no reescalarlas.

B. Paradigma de aprendizaje

El problema corresponde al paradigma de aprendizaje supervisado, debido a que el conjunto de datos contiene una variable objetivo conocida (Class) para cada transacción. El objetivo es utilizar las variables de entrada Time, V1 a V28 y Amount para aprender una relación entre las características de las transacciones y su respectiva clase, y utilizar posteriormente dicho modelo para clasificar nuevas transacciones como legítimas o fraudulentas.

Dado el fuerte desbalance de clases descrito en la Tabla I, la métrica principal para la evaluación de los modelos será el área bajo la curva de Precision-Recall (AUC-PR), en lugar del AUC-ROC, ya que este último puede resultar artificialmente alto bajo un desbalance tan extremo.

C. Estado del arte

La detección de fraude en transacciones con tarjetas de crédito ha sido abordada mediante diferentes técnicas de aprendizaje automático, principalmente debido al fuerte desbalance entre las transacciones legítimas y fraudulentas. A continuación se describen cuatro trabajos relevantes y su aporte específico al presente proyecto.

Du et al. [2] proponen el método AE-XGB-SMOTE-CGAN, que combina un autoencoder, XGBoost y Conditional GAN para abordar el desbalance de clases, alcanzando un MCC de 0,8845. Este trabajo es relevante porque utiliza la métrica MCC, apropiada para el fuerte desbalance de nuestro problema, y evidencia que las técnicas de generación sintética como SMOTE pueden combinarse con otras estrategias para mejorar la detección de la clase minoritaria.

El Coeficiente de Correlación de Matthews (MCC) se define como:

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{1}$$

donde TP, TN, FP y FN corresponden a los verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos, respectivamente. Esta métrica resulta especialmente adecuada en problemas con fuerte desbalance de clases, como el presente caso, ya que considera las cuatro categorías de la matriz de confusión de forma balanceada.

Carcillo et al. [3] presentan SCARFF, un *framework* de detección de fraude en flujos de transacciones basado en Kafka, Spark y Cassandra. A diferencia de los demás trabajos citados, no utiliza el conjunto de datos de 284.807 transacciones empleado en este proyecto, sino un flujo de datos industrial de mayor escala. Su aporte principal para este trabajo es metodológico: evidencia la importancia de considerar el desbalance de clases y los cambios de comportamiento en escenarios de detección de fraude.

Ileberi et al. [4] utilizan un Algoritmo Genético para la selección de características sobre el mismo conjunto de datos de 284.807 transacciones, obteniendo con Random Forest una Accuracy de 99,98% y una Precision de 95,34%. Este trabajo es particularmente relevante porque plantea una estrategia de reducción de variables aplicable a nuestro problema, a diferencia del PCA tradicional, dado que las variables V1-V28 ya son componentes de PCA.

Albalawi y Dardouri [5] comparan modelos tradicionales y de aprendizaje profundo sobre el mismo conjunto de datos, utilizando SMOTE y validación cruzada estratificada; su mejor modelo tradicional (XGBoost) obtuvo un F1-score de 93,30% y un ROC-AUC de 99,98%. Este trabajo aporta una referencia directa de desempeño para comparar los modelos que se entrenarán en el presente proyecto, al utilizar exactamente el mismo conjunto de datos.

TABLE III
RESUMEN COMPARATIVO DEL ESTADO DEL ARTE

Ref.	Técnica	Métrica	Aporte al proyecto
[2]	AE-XGB-SMOTE-CGAN	MCC = 0,8845	Uso de MCC para desbalance extremo
[3]	Framework streaming (Kafka/Spark)	Precision-recall	Importancia del desbalance y cambio de comportamiento
[4]	GA + Random Forest	Acc. 99,98%, Prec. 95,34%	Alternativa de reducción de variables sin PCA
[5]	XGBoost + SMOTE	F1 93,30%, ROC-AUC 99,98%	Referencia directa de desempeño (mismo dataset)

En conjunto, estos trabajos confirman que el desbalance de clases es el principal reto del problema abordado, y que estrategias como SMOTE, la generación sintética y la selección de características son las más utilizadas. Dado que en nuestro caso las variables ya corresponden a componentes de PCA, la reducción de dimensión del presente proyecto se abordará considerando esta particularidad, en lugar de aplicar PCA de forma directa como en los trabajos citados.

D. Configuración experimental

El conjunto de datos se dividió en entrenamiento (80%) y prueba (20%) de forma estratificada, conservando la misma proporción de fraudes en ambos subconjuntos, con semilla fija (`random_state=42`) para que la partición sea reproducible. Los registros duplicados se eliminaron antes de hacer la división, no después, para evitar que una misma transacción quedara repetida en entrenamiento y en prueba.

El conjunto de prueba se mantuvo completamente aparte durante toda la etapa de selección de modelos e hiperparámetros, y solo se usó una vez, al final, con los hiperparámetros ya definidos.

Para la validación durante el entrenamiento se usó validación cruzada estratificada de 5 particiones sobre el conjunto de entrenamiento. El escalado de `Time` y `Amount` se incluyó dentro de un `Pipeline`, de modo que la media y la desviación se calculan solo con los datos de entrenamiento de cada partición, sin filtrar información de la partición de validación. Las variables `V1` a `V28` no se volvieron a escalar, por las razones expuestas en la Sección II-A.

Para el desbalance de clases se evitó el uso de SMOTE puro, dado que con un 0,17% de fraudes la generación masiva de ejemplos sintéticos puede deformar la frontera entre clases. En su lugar, la regresión logística y Random Forest usan ponderación de clases (`class_weight="balanced"`), mientras que KNN, la red neuronal y la SVM usan submuestreo de la clase mayoritaria (relación 1:20) dentro de cada partición de entrenamiento, porque estos modelos no admiten ponderación de clases de forma nativa o su costo computacional con los 227 mil registros de entrenamiento resultaba inviable.

Se compararon cinco modelos, uno por cada familia exigida: regresión logística (paramétrico), KNN (no paramétrico), Random Forest (ensamble de árboles), un perceptrón multicapa (red neuronal) y una máquina de vectores de soporte. Cada uno se ajustó mediante búsqueda en malla sobre sus hiperparámetros principales, optimizando por AUPRC en validación. La Tabla IV resume los valores explorados para cada modelo.

TABLE IV
HIPERPARÁMETROS EXPLORADOS POR MODELO

Modelo	Hiperparám.	Valores evaluados (mejor en negrita)
Reg. logística	C	0,001; 0,01; **0,1**; 1; 10; 100
KNN	n_neighbors	11; 21; 51; 101; **201**
	weights	uniform; **distance**
Random Forest	n_estimators	100; **200**
	max_depth	8; **14**
Red neuronal	hidden_layer_sizes	(32,); **(64, 32)**; (64,); (128, 64)
	alpha	0,0001; 0,01; **0,1**
SVM	kernel	**linear**; rbf
	C	**0,01**; 0,1; 1; 10

En cuatro de los cinco modelos el mejor valor encontrado quedó en el borde de la malla explorada: KNN (k=201), Random Forest (max_depth=14), la red neuronal (alpha=0,1) y la SVM (C=0,01). Solo en la regresión logística el mejor valor (C=0,1) quedó dentro del rango, lejos de sus extremos. Esto indica que, para los otros cuatro modelos, un valor fuera del rango explorado podría mejorar aún más el resultado; este punto queda como trabajo pendiente para una siguiente iteración, ampliando la malla en esas direcciones.

Como métrica principal se usó el área bajo la curva Precision-Recall (AUPRC), en lugar de Accuracy o ROC-AUC, porque con un desbalance tan extremo un modelo sin ninguna capacidad predictiva ya obtiene una Accuracy cercana al 99,8% y un ROC-AUC engañosamente alto. Como referencia, el AUPRC de un modelo sin información es igual a la proporción de fraudes en los datos (0,0017), así que cualquier modelo útil debe superar ampliamente ese valor. De forma complementaria se calcularon también ROC-AUC, Precision, Recall, F1 y MCC (definido en la Ecuación 1), evaluados con el umbral de decisión por defecto (0,5).

III. RESULTADOS Y DISCUSIÓN

Las Figuras 1 y 2 muestran el efecto de cada hiperparámetro sobre el AUPRC de entrenamiento y validación. En la regresión logística, el AUPRC se estabiliza a partir de $C = 0{,}01$. En KNN, la ponderación por distancia (*distance*) mejora notablemente el AUPRC de entrenamiento a medida que crece k, pero esto no se traduce en una mejora proporcional en validación, lo que sugiere sobreajuste con valores altos de k y esa ponderación. En Random Forest, tanto la profundidad como el número de árboles mejoran el AUPRC de forma consistente en ambos conjuntos, sin señales claras de sobreajuste adicional dentro del rango explorado. En la red neuronal, las arquitecturas más grandes, (64, 32) y (128, 64), superan claramente a las más pequeñas.

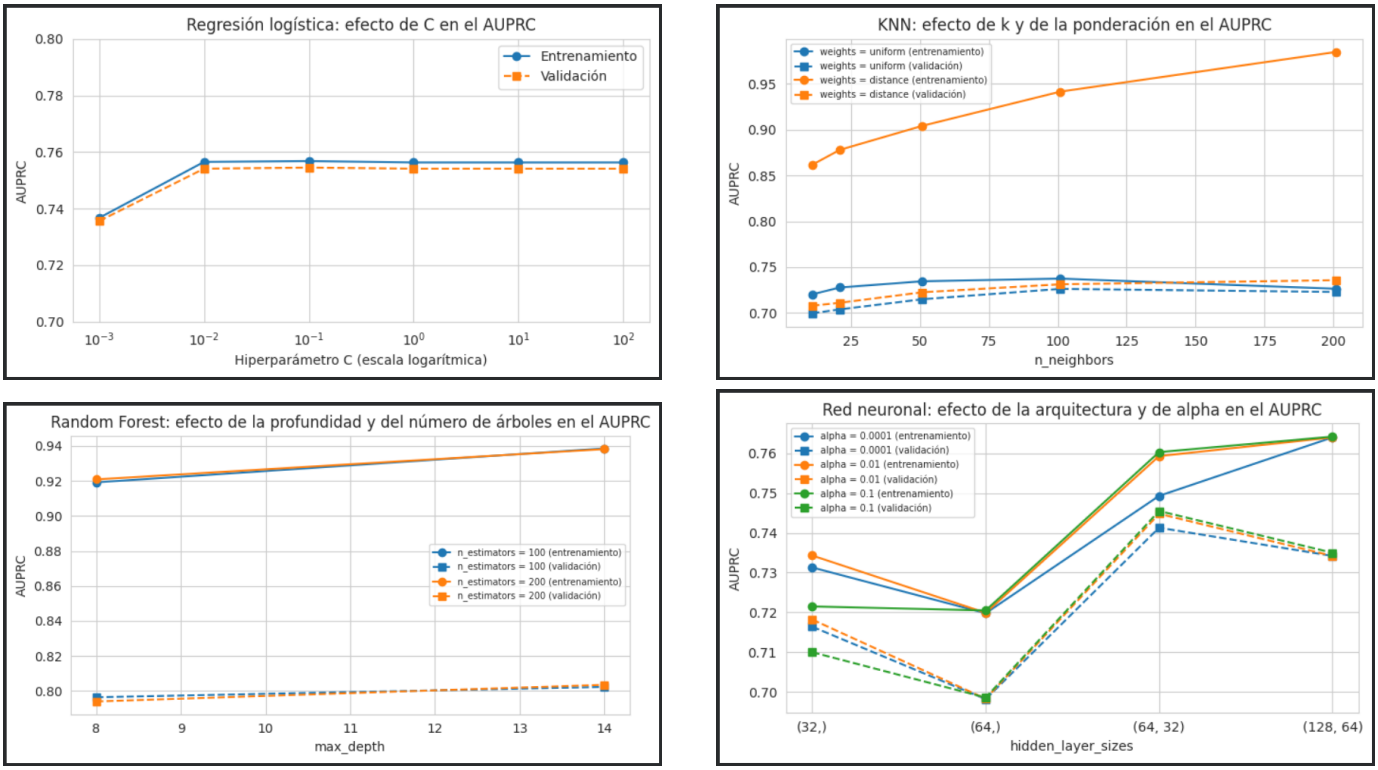


Fig. 1. Efecto de los hiperparámetros en el AUPRC de entrenamiento y validación. De izquierda a derecha y de arriba a abajo: regresión logística, KNN, Random Forest y red neuronal.

En la SVM, el kernel RBF con $C = 1$ alcanza el mejor AUPRC de validación, pero cae notablemente con $C = 10$, mientras que el kernel lineal se mantiene estable en todo el rango.

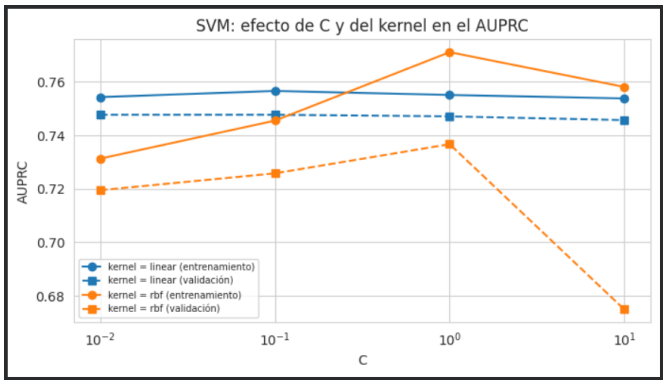


Fig. 2. Efecto del hiperparámetro C y del kernel en el AUPRC de entrenamiento y validación para la SVM.

Dado que la SVM utilizó submuestreo de la clase mayoritaria en lugar de ponderación de clases, se entrenó una versión adicional con `LinearSVC` y `class_weight="balanced"` sobre el conjunto de entrenamiento completo, para comparar ambas estrategias de manejo del desbalance. Esta versión alcanzó un AUPRC en validación de 0,7393, ligeramente inferior al 0,7477 obtenido con submuestreo, lo que indica que, para este modelo y este problema, el submuestreo resultó más efectivo que la ponderación de clases.

La Tabla V resume el AUPRC obtenido por cada modelo en entrenamiento, validación (promedio de 5 particiones, con intervalo de confianza del 95%) y prueba (intervalo de confianza del 95% mediante bootstrap con 1000 remuestreos sobre las transacciones de prueba).

TABLE V
AUPRC POR MODELO EN LOS TRES CONJUNTOS

Modelo	**Entren.**	**Validación (IC 95%)**	**Prueba (IC 95%)**
Random Forest	0,9382	0,8035 ± 0,0436	0,7833 [0,7007; 0,8691]
Reg. logística	0,7568	0,7545 ± 0,0355	0,6852 [0,5825; 0,7855]
SVM	0,7543	0,7477 ± 0,0601	0,7072 [0,6030; 0,8022]
Red neuronal	0,7603	0,7455 ± 0,0450	0,7092 [0,5972; 0,8055]
KNN	0,9849	0,7358 ± 0,0332	0,6766 [0,5678; 0,7820]

Random Forest obtuvo el mejor AUPRC tanto en validación (0,8035) como en prueba (0,7833), y es también el único modelo con una diferencia marcada entre entrenamiento (0,9382) y

validación (0,8035), lo que sugiere cierto grado de sobreajuste. En KNN, el AUPRC de entrenamiento (0,9849) tampoco es comparable con el de los demás modelos: al evaluarse sobre los mismos datos con los que se entrenó, cada transacción se encuentra a sí misma como su vecino más cercano, lo que infla artificialmente esa cifra.

En validación, la regresión logística, la SVM y la red neuronal quedaron prácticamente empatadas, entre 0,7455 y 0,7545, con intervalos de confianza solapados entre sí; KNN quedó algo por debajo (0,7358). Esto indica que, en validación, no hay evidencia suficiente para afirmar que alguno de esos tres modelos sea mejor que los otros dos.

En prueba, el orden cambió de forma notable: la red neuronal (0,7092) y la SVM (0,7072) superaron a la regresión logística (0,6852), cuyo resultado en prueba quedó por debajo de su propio intervalo de validación. Sin embargo, los intervalos de confianza en prueba son muy amplios para los cinco modelos (entre 0,10 y 0,20 de ancho), debido al tamaño reducido del conjunto de prueba (95 fraudes). Esta amplitud hace que ninguna de las diferencias entre los modelos que no son Random Forest resulte concluyente: sus intervalos se solapan ampliamente tanto entre sí como con el de Random Forest.

Un aspecto importante a considerar es el umbral de decisión usado para las métricas Precision, Recall, F1 y MCC. Inicialmente se evaluaron con el umbral por defecto (0,5), lo que perjudicaba fuertemente a la regresión logística (Precision de 0,0563 y F1 de 0,1059 en prueba), a pesar de que su AUPRC era comparable al de los demás modelos. Esto se debe a que la regresión logística, al no usar `predict_proba` sino `decision_function`, produce puntuaciones en una escala muy distinta a 0,5, haciendo que ese umbral fijo no tuviera sentido para este modelo en particular.

Para corregir esto, se seleccionó un umbral específico para cada modelo maximizando F1 sobre predicciones fuera de pliegue (*out-of-fold*) obtenidas por validación cruzada sobre el conjunto de entrenamiento, sin usar en ningún momento el conjunto de prueba para esta elección. Al aplicar este umbral ajustado, la regresión logística alcanzó una Precision de 0,8765, un F1 de 0,8068 y un MCC de 0,8091 en prueba, resultados comparables a los de los demás modelos. Con el umbral ajustado, los cinco modelos obtuvieron un F1 entre 0,80 y 0,81, lo que confirma que, descontando el efecto del umbral, su capacidad de clasificación es muy similar; la diferencia principal entre ellos sigue estando en el AUPRC, donde Random Forest mantiene la ventaja.

REFERENCES

[1] Kaggle, "Credit Card Fraud Detection," Machine Learning Group - ULB. [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[2] H. Du, L. Lv, H. Wang, and A. Guo, "A novel method for detecting credit card fraud problems," *PLoS ONE*, vol. 19, no. 3, e0294537, 2024.

[3] F. Carcillo, A. Dal Pozzolo, Y.-A. Le Borgne, O. Caelen, Y. Mazzer, and G. Bontempi, "Scarff: a scalable framework for streaming credit card fraud detection with Spark," *Information Fusion*, vol. 41, pp. 182-194, 2018.

[4] E. Ileberi, Y. Sun, and Z. Wang, "A machine learning based credit card fraud detection using the GA algorithm for feature selection," *Journal of Big Data*, vol. 9, art. 24, 2022.

[5] T. Albalawi and S. Dardouri, "Enhancing credit card fraud detection using traditional and deep learning models with class imbalance mitigation," *Frontiers in Artificial Intelligence*, vol. 8, art. 1643292, 2025.